

Install Python as a class

We are going to go through this slowly as a class

Some common nomenclature

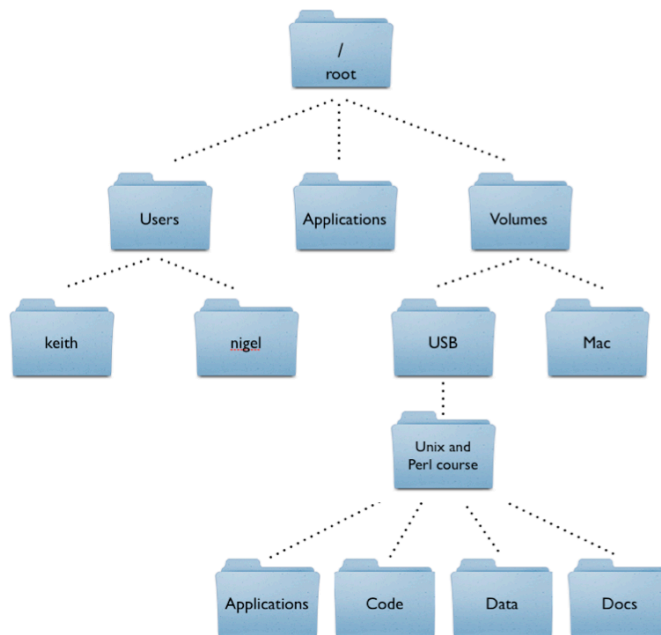
- pwd or dir on windows - tells you the path or directory you are in
- ls - lists everything in that folder or directory you are in
- cd - change directory. Lets you change a directory
 - cd name changes directory into that folder
 - cd .. moves you up a directory
 - cd / moves you to root or the top of all your folders

Folder or directory structure

```
In [3]: from IPython.display import Image
```

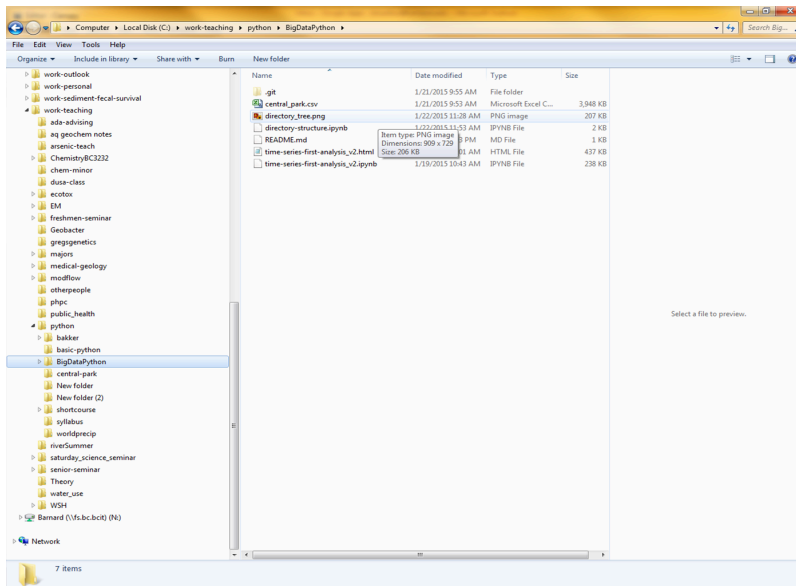
```
In [7]: Image(filename='directories/directory_tree.png',width=400)
```

Out[7]:



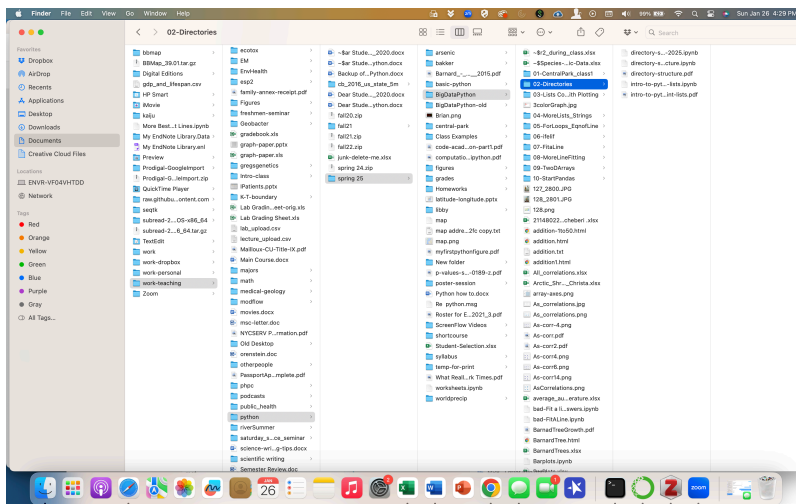
```
In [8]: Image(filename='directories/PC.png',width=400)
```

Out [8]:



In [9]: `Image(filename='mac-files.png',width=400)`

Out [9]:



Installation

1. Open Finder or explorer and go to your working directory
2. Open Terminal or Prompt. Do not change directories yet.
3. Install UV
 - A. go to <https://docs.astral.sh/uv/getting-started/installation/> (you can google install uv python)
 - B. Install UV. I do curl. copy and paste the command
4. In terminal or prompt go to your working directory
5. Now Install Python with UV
 - A. `uv init --python 3.11`
 - B. `uv add matplotlib pandas jupyterlab`
6. Now to run python
 - A. `"uv run jupyter lab"`
7. If jupyter lab opens in a browser pat yourself on the back and take a deep breath

Now let's test it all.

1. Exit Jupyter lab.
2. Close terminal and finder.
3. Look in Finder or explorer in your working directory. Are there some new files?
4. open terminal. type `cd` and then drag in your working directory to change directories.
5. in your working directory type `"uv run jupyter lab"`
6. If it works again we are ready!

When running Python in Jupyter lab

- Always start in the directory where you want to work.

Now let's learn some basic Python

First we start with Markdown

Look up toward the top where it says code. Click that to Markdown. Now you can add notes

We are using Markdown which is great for making notes and can be used to do math notation also.

This is H1 one `#` gives you H1 You type `# H1` at the beginning of the line

this is H2 two `##` gives you H2

this is h3

you can also use html notation which you have to click on the markdown to see!
So click on this cell to see how it is done. See above how this cell is Markdown. This allows us to add notes!

To see how this is done. Go to Github. Download the python notebook. Open it. Then double click on this. Remember to download I right click on raw and then do save as. It seems to work smoothest in chrome.

If you put one `*` before and after a word you get *italic* if you put two you get **Bold** and three you get both ***Bold and Italic***

You can add an equation

$$c = \sqrt{a^2 + b^2}$$

here is a link to a cheat sheet.

<https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet>

I can also make the link inline

To make it inline you put the text in brackets [] and then the link next to it in () click to see

[Here is a link to a Latex cheat sheet.](#) any search engine will be able to help you.

Now we can do some math!

I am going to throw some programming tricks at you!

An Aside

I put all my notebooks on github. It is a version control program. It is like dropbox but keeps track of changes. On the class webpage are all the links to the notebooks on github.

to get the notebook is a little tricky. Some web browsers are easier than others. I like chrome. In chrome after getting to the website I right click on raw and then choose save as. You should now be able to save as an .ipynb file and have exactly what I hand out.

Do not copy and paste from the online notebook. I find people who copy and paste do not learn the muscle memory and do worse in class. I give you time each day to work through the notebooks. Type it in and work through each notebook

In []:

```
In [10]: 2+2
```

```
Out[10]: 4
```

```
In [12]: 3*4
```

```
Out[12]: 12
```

```
In [13]: 6/4
```

```
Out[13]: 1.5
```

this is New!!!! Python 3 does integer math correct!

```
In [14]: type(6)
```

```
Out[14]: int
```

but what if we have a decimal?

```
In [15]: type(6.0)
```

```
Out[15]: float
```

```
In [16]: 6.0/4
```

```
Out[16]: 1.5
```

If you wanted to use integers you use //

```
In [17]: 6.0//4
```

```
Out[17]: 1.0
```

how python treats decimals is very different between Python 2 and 3. Python 2 used to default to integers. Python 3 no longer does this. It is nice!

I realize many of you haven't thought about floats and ints. Here is an overview showing why integers take less [memory](#) and are also good for looping and counting.

- integers are more like whole numbers and used for counting
- floats are numbers with decimals and used for math

```
In [18]: #but that is if we don't have a calculator. We can use parameters in program  
#this will move us towards something more powerful  
a=6  
b=4  
a/b
```

```
Out[18]: 1.5
```

```
In [19]: a%b #this is call mod and gives the remainder! Remember this!
```

```
Out[19]: 2
```

```
In [20]: #I almost forgot exponents  
a**2
```

```
Out[20]: 36
```

```
In [21]: a**0.5
```

```
Out[21]: 2.449489742783178
```

Remember computers always remember PEMDAS! Do you remember PEMDAS?

```
In [22]: 2+2**2
```

```
Out[22]: 6
```

```
In [23]: (2+2)**2
```

```
Out[23]: 16
```

```
In [24]: c=(2+2)**2  
print (c)
```

```
16
```

```
In [25]: c
```

```
Out[25]: 16
```

this is a bit of wacky python that is helpful. You can have more than one thing on each side of an equal sign

```
In [26]: a,b=1,2  
print(a,b)
```

```
1 2
```

```
In [27]: a,b,c=1,2,3  
print(a,b,c)
```

```
1 2 3
```

```
In [28]: a,b,c='1','hello',2  
print(a,b,c)
```

```
1 hello 2
```

```
In [29]: a,b,c='1','hello',[5,5,6]
```

```
print(a,b,c)
```

1 hello [5, 5, 6]

```
In [30]: import numpy as np #more to come on this!
a,b,c='1','hello',np.arange(1,10)
print(a,b,c)
```

1 hello [1 2 3 4 5 6 7 8 9]

Write down your answer to

$$7 + (6 \times 5^2 + 3)$$

then try it!

In []:

What about that dumb [viral math problem](#)?

$$8 \div 2(2 + 2)$$

In [11]:

Out[11]: 16.0

Now to some printing....

- you put text in quotes.
- you put variables by themselves
- you separate by columns
- '\n' adds a new line

```
In [31]: a=6
b=4
print ('The result of a/b=',a/b)
```

The result of a/b= 1.5

```
In [32]: print ('we can get fancier \na=',a,'\nb=',b,'\na/b=',a/b) #the \n is a new
#you can also to \t for
```

we can get fancier
a= 6
b= 4
a/b= 1.5

```
In [33]: output='we can get fancier\na='  
         #I just set the parameter output to the writing  
         # and then called it later on.  
         print (output,a)
```

```
we can get fancier  
a= 6
```

```
In [34]: output='we can get fancier\na='  
         print (output,float(a)) # we just cast a to a float
```

```
we can get fancier  
a= 6.0
```

```
In [35]: output
```

```
Out[35]: 'we can get fancier\na='
```

```
In [36]: str(output)
```

```
Out[36]: 'we can get fancier\na='
```

Fancy Formating

Try to always use this

The next set of calls shows you the formal way of formatting all your values. Plus I added the "\n" (the slash doesn't always show up) to do a line break below. Play with all the settings. The format is an object oriented function you are calling to set the formatting. This [link](#) goes through some string stuff and this [link](#) gets into the nitty-gritty detail of how to do the formatting. You can set widths, spacing, decimals, and centering. Go through and do some of your own! Go through this slowly. We will be using it a lot to make things nice. You will come back to this!

Take your time here and don't rush. It will save you time later on making things look nice

First example

```
In [37]: a=6  
         print('The value of a is {}'.format(a))
```

```
The value of a is 6
```

You add {} where you want to print the variable and put the variable in the format statement

```
In [38]: a=6  
         print('The value of a is {} and I can repeat a={}'.format(a,a))
```

The value of a is 6 and I can repeat a=6

You can add multiple variables

```
In [39]: a=6
b=4
print('The value of a is {} and b is={}'.format(a,b))
```

The value of a is 6 and b is=4

You add line breaks

```
In [40]: a=6
b=4
print('The value of a is {}\nb is={}'.format(a,b))
```

The value of a is 6

b is=4

Be careful because you could do this.

```
In [41]: a=6
b=4
print('The value of a is {}\nb is={}'.format(b,a))
```

The value of a is 4

b is=6

Now we can start to format the numbers.

- d is for integers
- f is for floats
- you can also do strings but I don't use that much

```
In [42]: print ('The value of b is {:3d} and the value of a is {:.3f}'.format(a,b))
# or you can just not print the decimals for a float
# The 3 in front of the d is for the number of spaces.
#The .3 in front of the f is the number of decimal places.
#The total number of spaces would be in front of
# the decimal place.
```

The value of b is 6 and the value of a is 4.000

```
In [43]: #lets do more fomattting. If you want 2 #the slash allows us to split up a
print ('we are going to divide {} by {}'.format(a,b))
print ('we could turn {} into a float {:.3f}'.format(a,a))
print ('we could turn {} into a float with 10 decimal places \
and 15 spots and a plus sign with stars and centered {:*^15.10f}'.format(a,
```

we are going to divide 6 by 4

we could turn 6 into a float 6.000

we could turn 6 into a float with 10 decimal places and 15 spots and a plus sign with stars and centered `++6.000000000000*`

```
In [44]: #fancy formatting.
print ('The value of b is {:*^14.3f}\nthe value of a is{:3d}'.format(b,a))
```

```
#The 0 represents the first item in the list. The star represents the fill.  
# the carrort says center it.  
# The 14 represents the spaces.  
# The 3 repesents the decimals. the f is a float
```

The value of b is ****4.000****
the value of a is 6

You can also do Math in the format statement

```
In [45]: print ('The value of b is {}\nthe value of a is{}\n  
and the value of b/a is {:.3f}'.format(b,a,b/a))
```

The value of b is 4
the value of a is 6 and the value of b/a is 0.667

I just made all the formatting more complicated then it needs to be to teach you. The key thing to remember is you can control the number of decimal places. That is really nice to make output look nice especially as we start reporting variables on graphs etc (this is foreshadowing, remember this).

I am sure I did a few things you missed. So a couple of notes for the HW

1. Set your printed decimal points

```
In [46]: a=3.14159265  
print('pi is equal to {:.3f}'.format(a))
```

pi is equal to 3.142

2. You can add line breaks wherever you want with \n to make things look nice (or ugly)

```
In [12]: a=3.14159265  
print('pi is equal to {:.3f}\nBut maybe I want 5 decimals: pi={:.5f}'.format(a))
```

pi is equal to 3.142
But maybe I want 5 decimals: pi=3.14159

3. a tab is \t and adds a nice space

```
In [47]: a=3.14159265  
print('pi\tis equal to {:.3f}'.format(a))
```

pi is equal to 3.142

Make sure to make a nice heading for your homework in Markdwon

Name

HW Number

You can then paste in the questions

In []:

In []:

new page

Exit Ticket

Name:

uni:

Q1. Write code to set a=6.70000 and then print to one decimal place using .format

code:

output: a=6.7

In []: